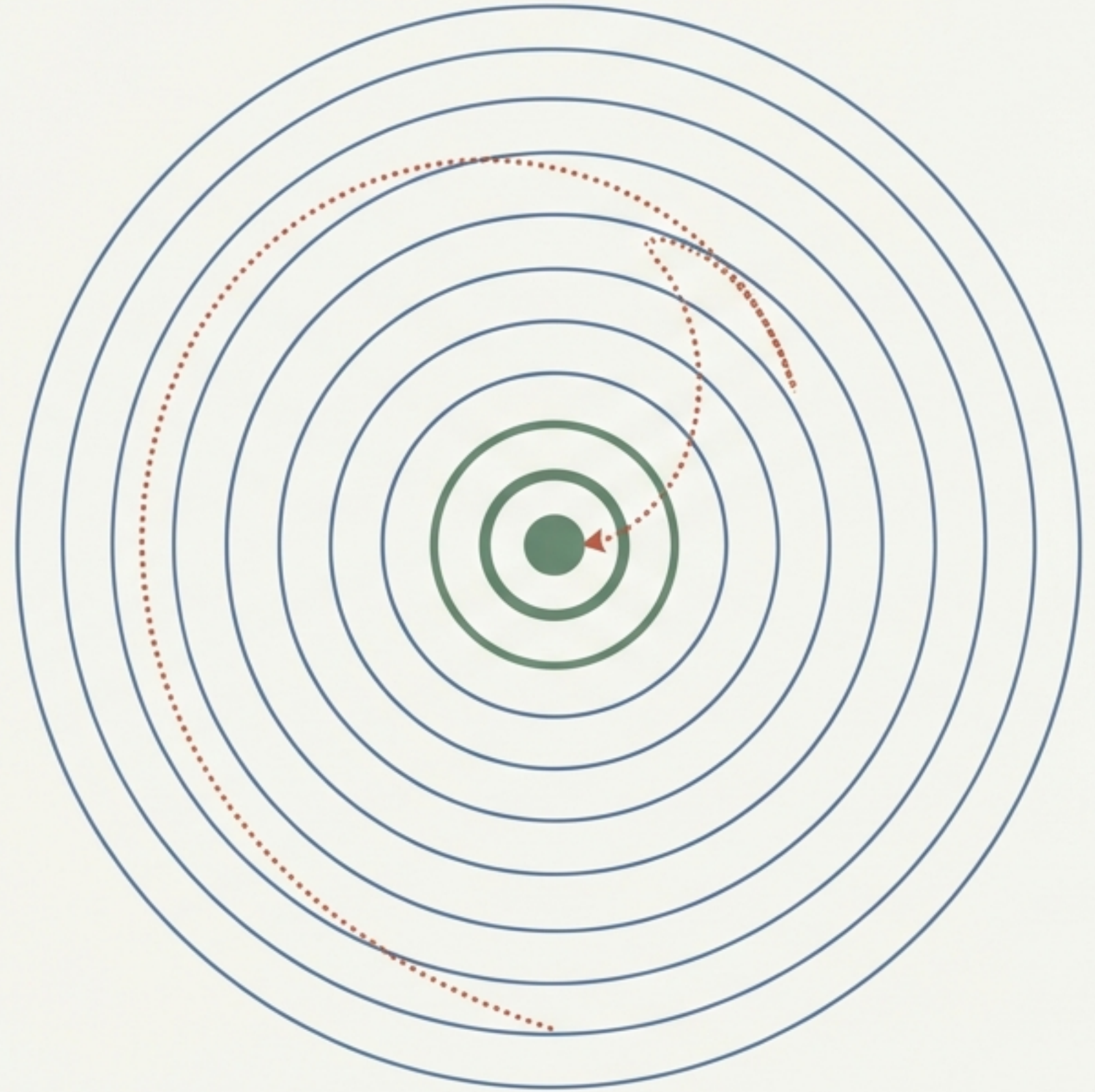


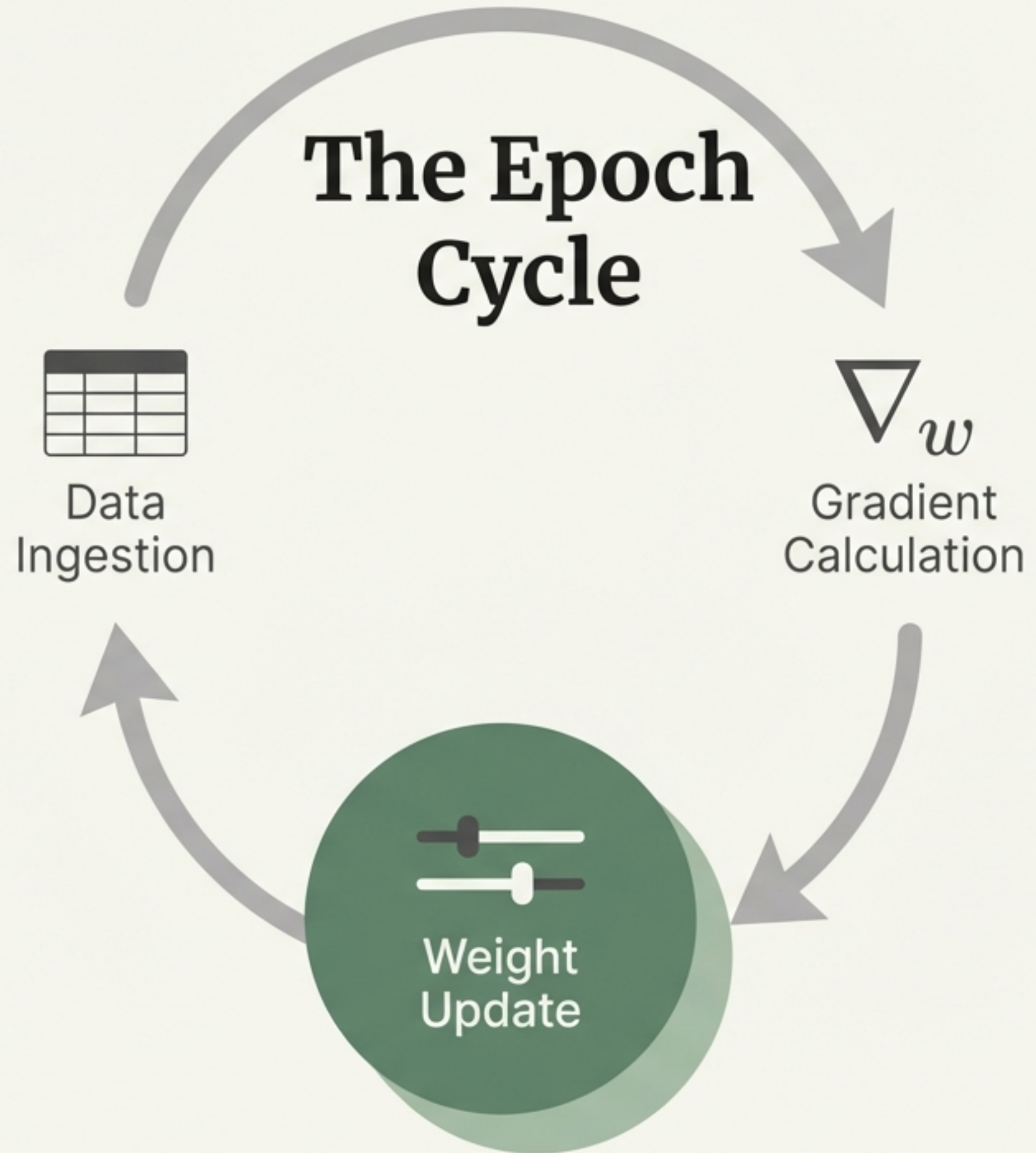
Mastering Mini-Batch Gradient Descent

The definitive guide to the sweet spot of optimisation algorithms—from underlying mechanics to from-scratch Python implementation.



The Core Differentiator: Update Frequency

All Gradient Descent variants share the same mathematical goal. They are separated by just one operational rule: **how frequently the algorithm updates the model weights** during a single epoch.



The Two Extremes of Optimisation



Batch Gradient Descent

- 1 update per epoch (processes all data simultaneously).
- Verdict: Highly stable, but computationally sluggish and unscalable for large datasets.

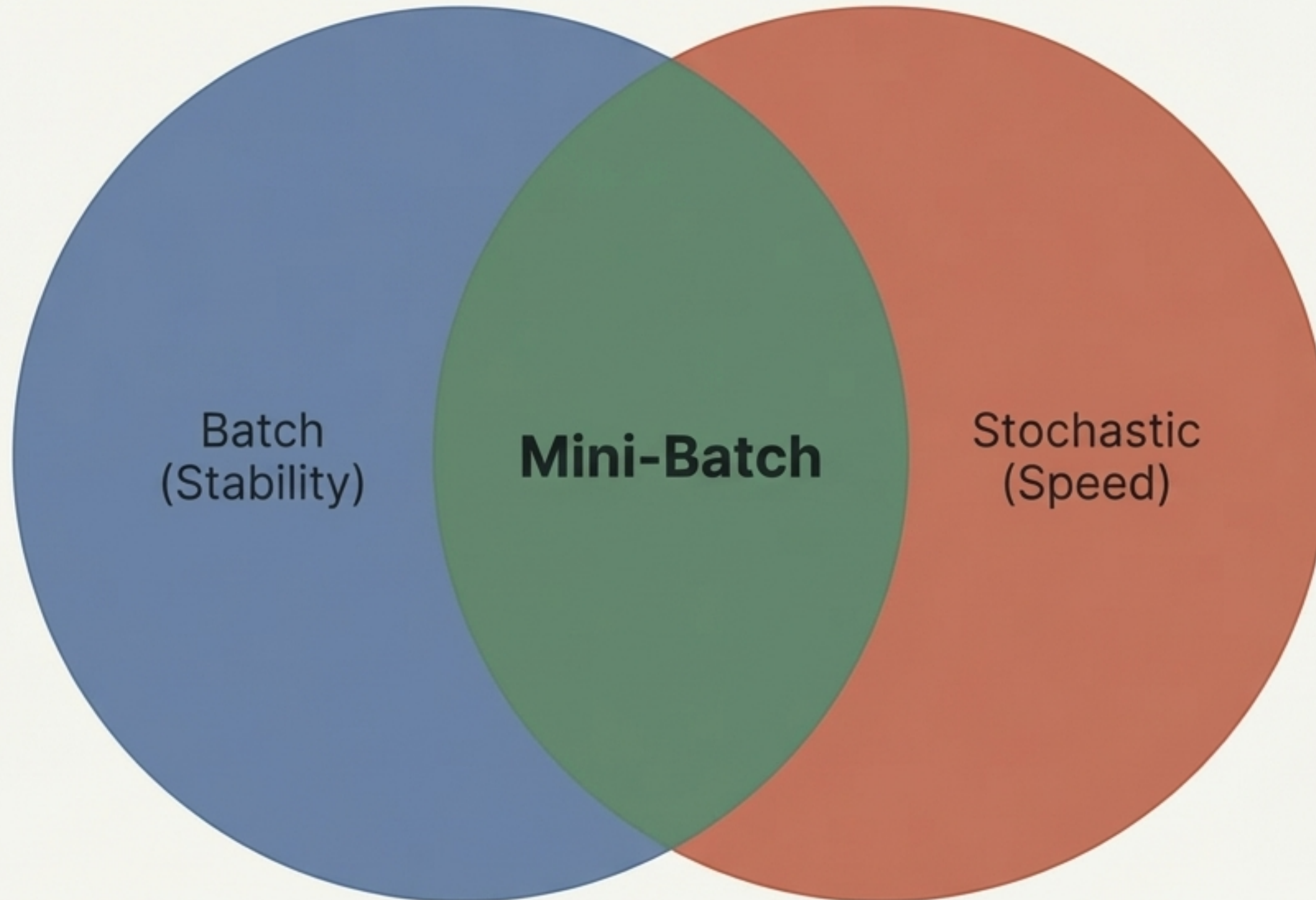


Stochastic Gradient Descent (SGD)

- N updates per epoch (1 update per single data row).
- Verdict: Lightning fast and escapes local minima, but highly chaotic. Rarely settles perfectly at the global optimum.

The Goldilocks Solution

Mini-Batch Gradient Descent captures the computational efficiency of the Batch approach while retaining the dynamic speed of Stochastic. It is the optimal middle ground.



The Mechanics of the Batch

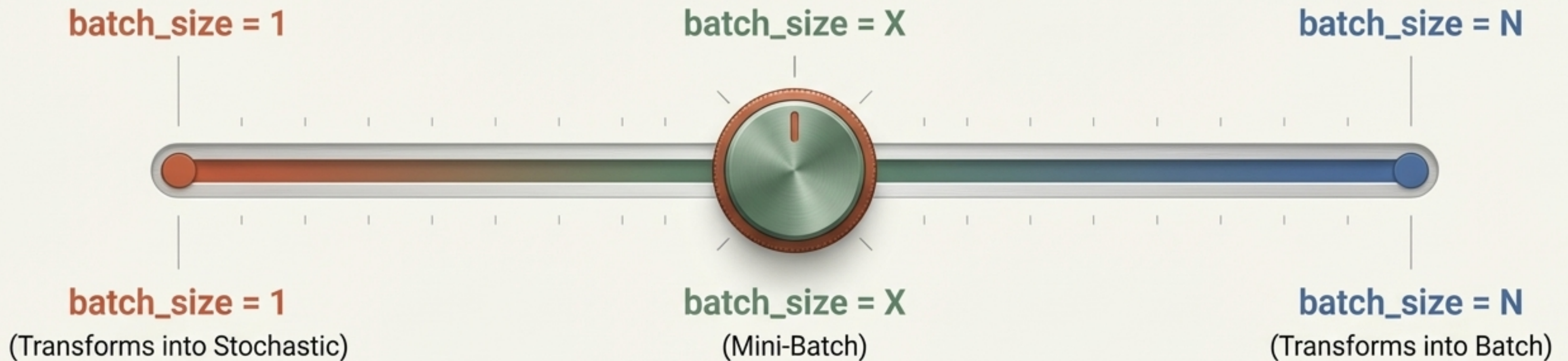
Instead of processing one row or all rows, the dataset is divided into uniform groups.

The Formula: With N total rows and a chosen `batch_size` of B , the algorithm performs N/B updates per epoch. (e.g., 1,000 rows / 100 batch size = 10 sequential updates per epoch).



The Parameter Spectrum

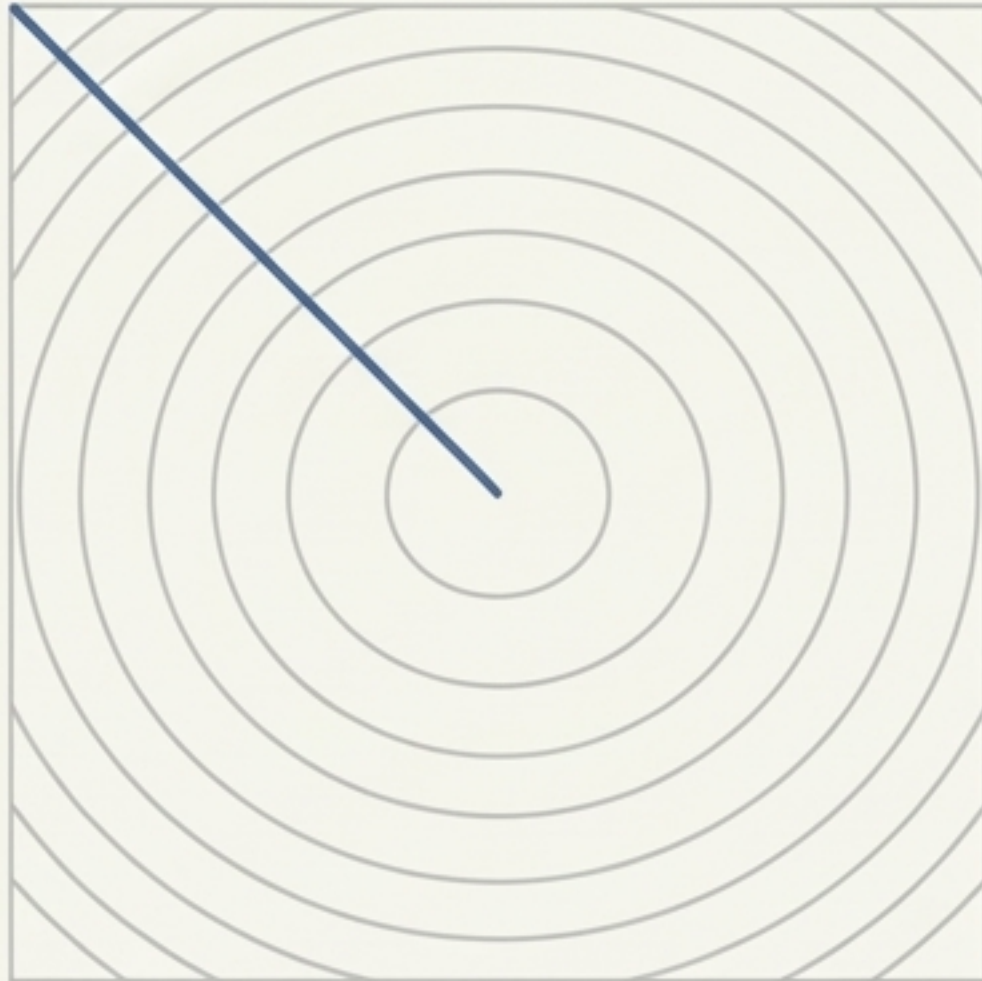
Mini-Batch Gradient Descent is highly tunable. By adjusting a single parameter—`batch_size`—you control the physical behaviour of the algorithm, sliding seamlessly between extreme stability and extreme randomness.



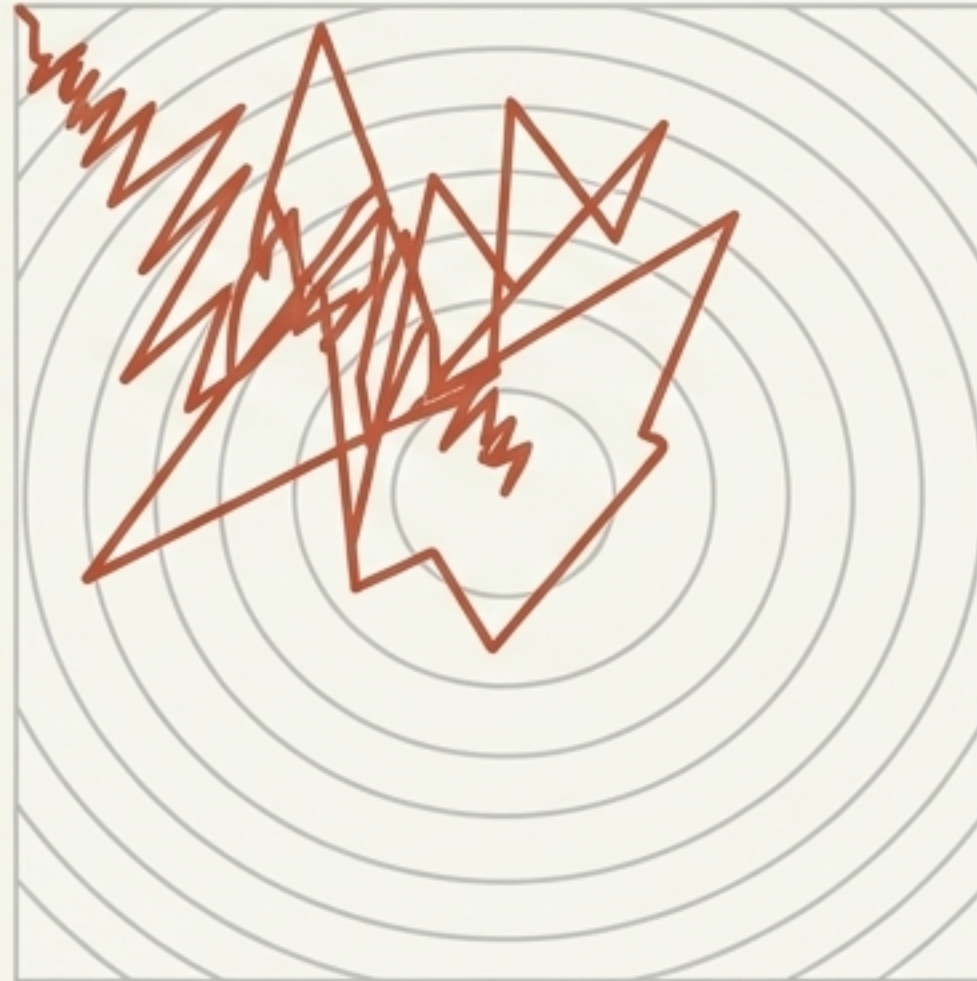
Visualising the Convergence Path

Mini-Batch exhibits a path that is neither entirely strict nor entirely random. It absorbs enough noise to escape local minima without losing its overall directional efficiency.

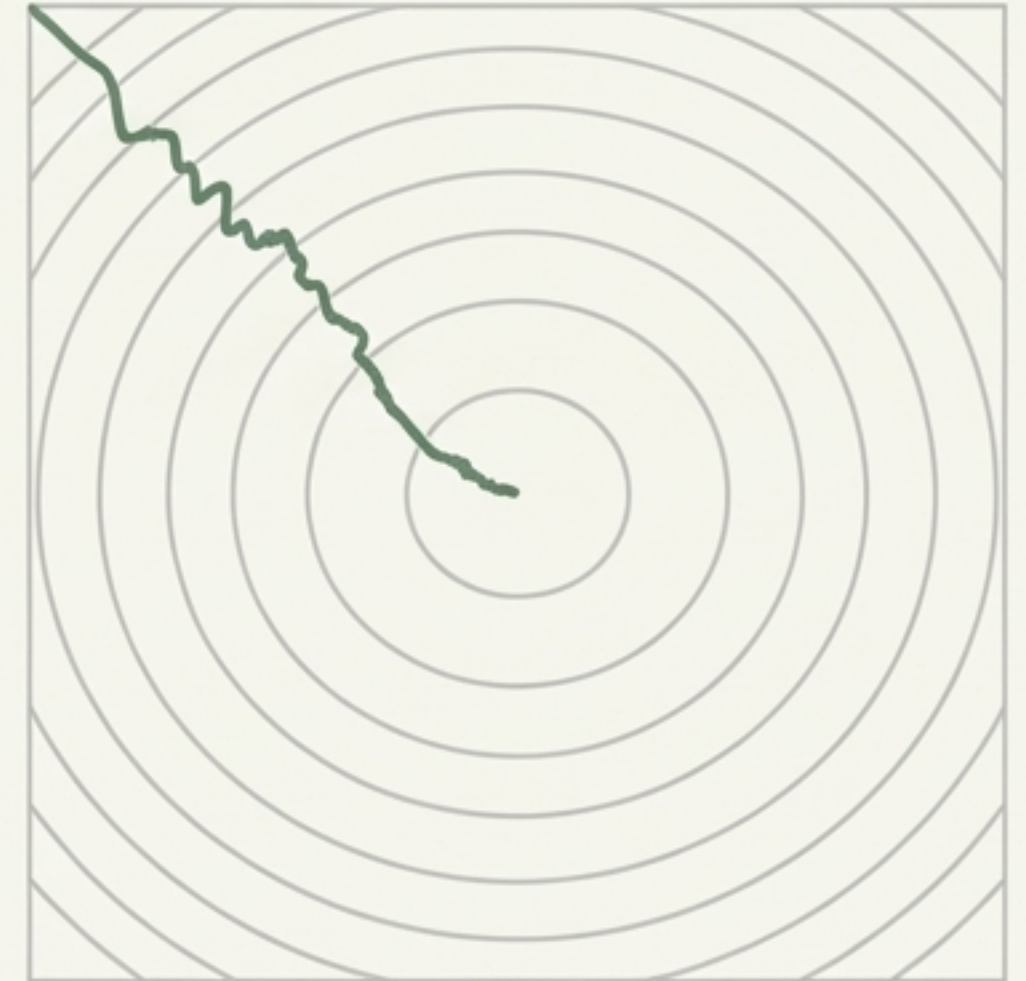
Batch



Stochastic



Mini-Batch

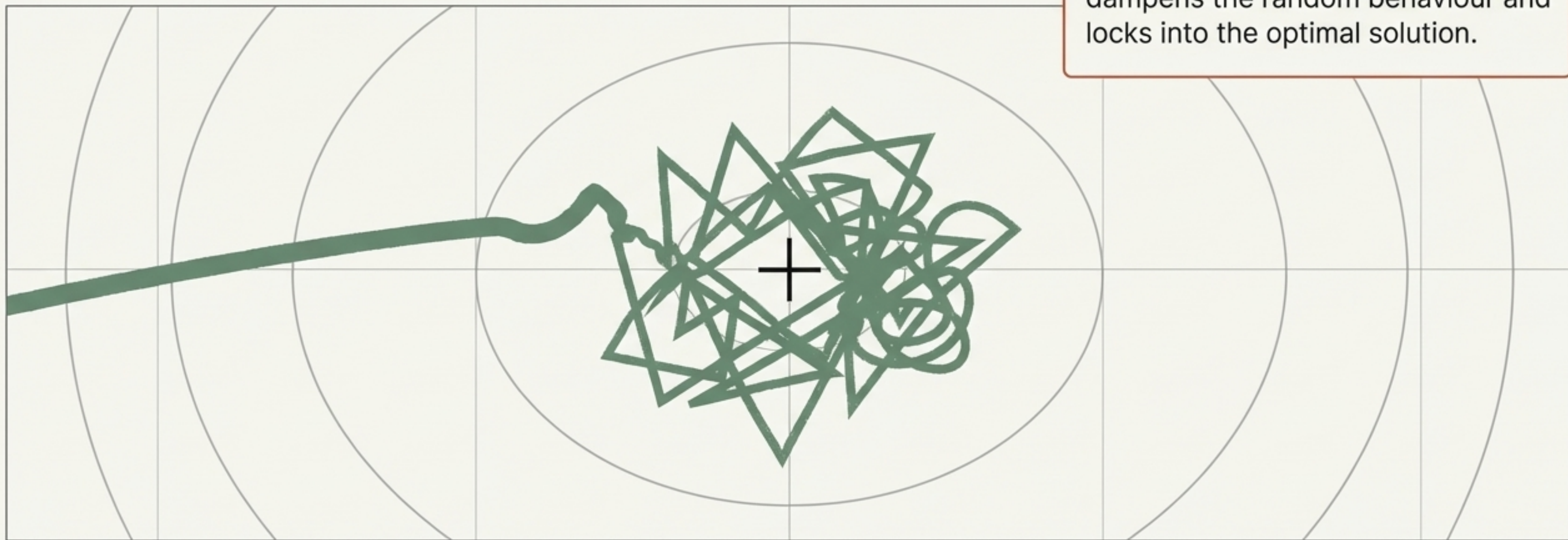


The Bouncing Problem

Because Mini-Batch retains a fraction of Stochastic's randomness, it can perpetually bounce around the global minimum rather than settling perfectly.

The Fix: Learning Schedules

Dynamically decay the learning rate as convergence approaches. This dampens the random behaviour and locks into the optimal solution.



Building from Scratch: Class Setup

To adapt a standard Gradient Descent class, we only need to introduce one new hyperparameter during initialization.

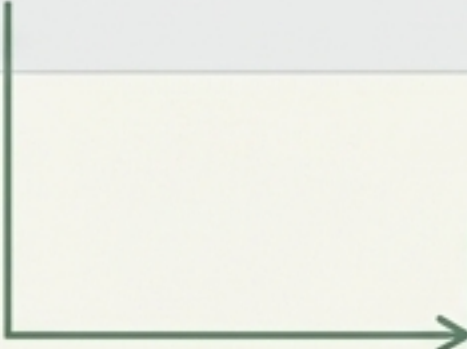


```
def __init__(self, batch_size, learning_rate, epochs):  
    self.batch_size = batch_size
```


Building from Scratch: The Batch Loop

The defining structural change requires nesting a batch loop inside the standard epoch loop. The inner loop runs exactly N/B times per epoch.

```
for i in range(epochs):  
    for j in range(int(X_train.shape[0] / self.batch_size)):
```

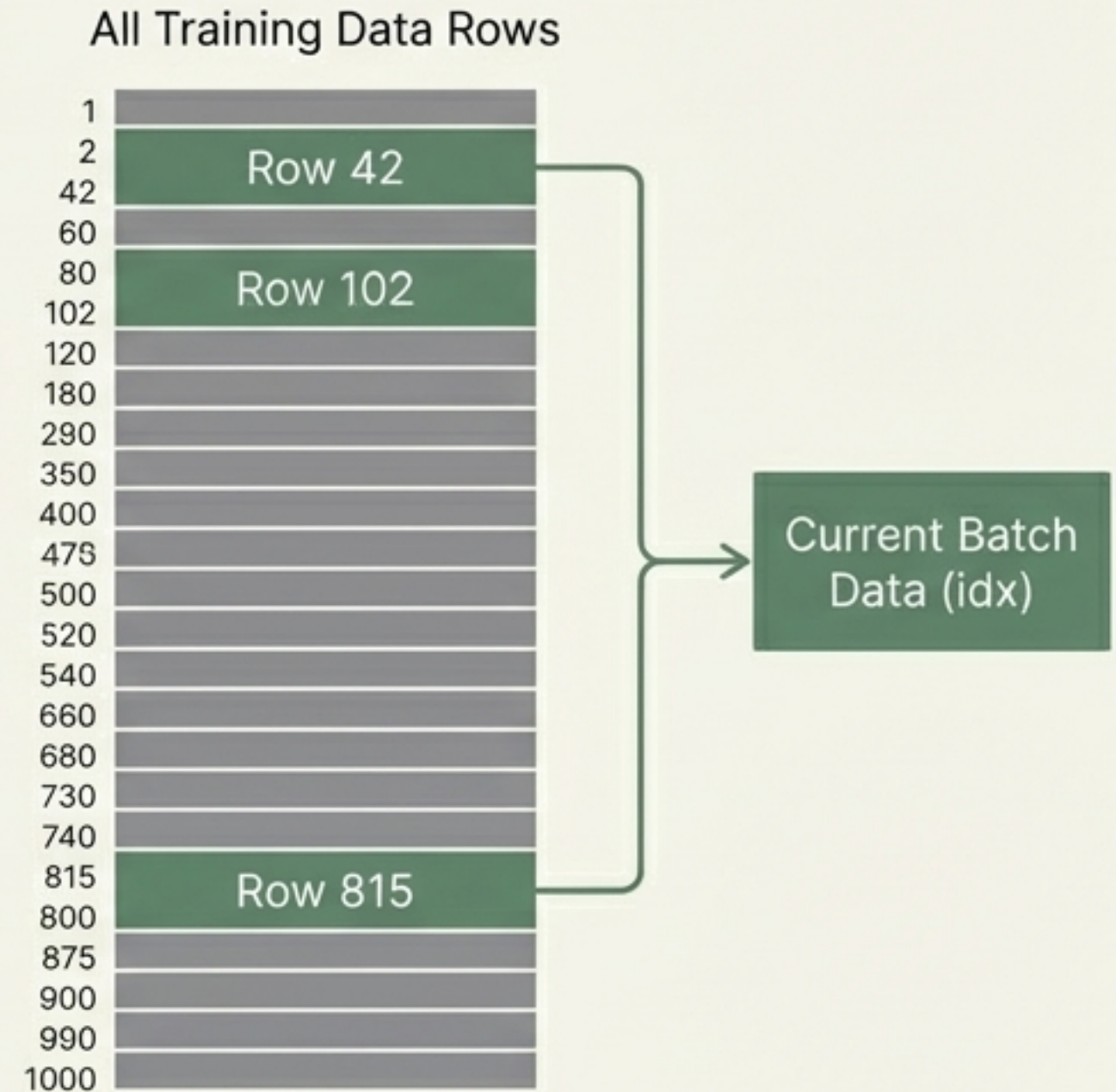


Total rows divided by batch size
determines total batches per epoch

Building from Scratch: Random Sampling

We must randomly select row indices for each batch to ensure the algorithm evaluates diverse data subsets and avoids sequence bias.

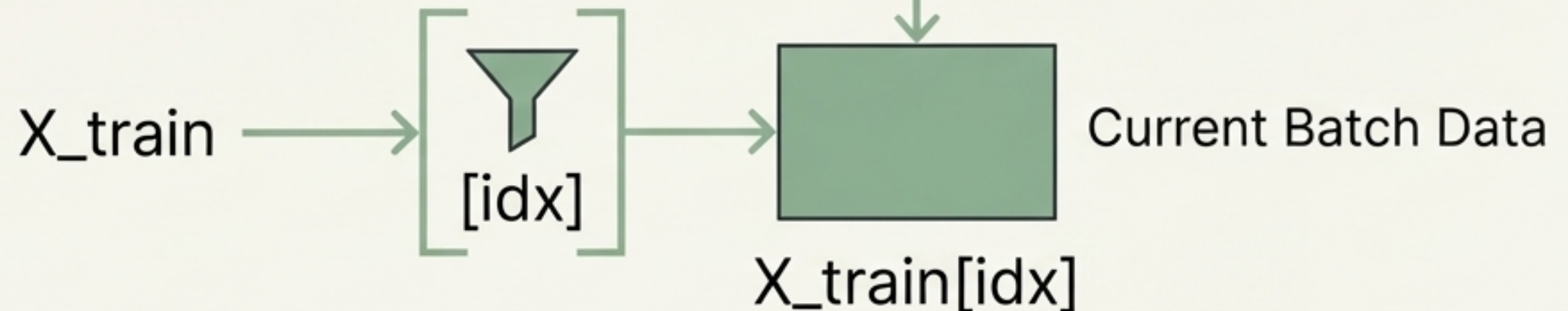
```
idx = random.sample(range(X_train.  
    shape[0]), self.batch_size)
```



Building from Scratch: The Update Step

Applying the mathematical derivatives exclusively to the isolated mini-batch subset. Intercepts and slopes are updated iteratively using only the data captured within the current idx array.

```
y_hat = np.dot(X_train[idx], self.coef_) + self.intercept_
```



The Scikit-Learn Dilemma

Unlike deep learning libraries, Scikit-Learn's native SGDRegressor does not feature a `batch_size` parameter. By default, its internal architecture enforces strict Stochastic Gradient Descent (`batch_size = 1`).

Keras PyTorch



Native Batch Support

Scikit-Learn



No Direct Batch Parameter

Forcing Scikit-Learn to Process Mini-Batches

The industry workaround requires manually feeding the model subsets of data step-by-step using the **partial_fit()** method instead of **.fit()**.

Bypasses standard fitting to allow manual batch ingestion.

```
sgd = SGDRegressor()  
for i in range(epochs):  
    # Generate random batch indices  
    sgd.partial_fit(X_train[idx], y_train[idx])
```



The Optimal Optimiser



The Sweet Spot

Perfectly balances computational stability with processing speed.



Highly Customisable

A single `batch_size` dial scales behaviour smoothly between Batch and Stochastic extremes.



The Industry Standard

The default, foundational optimisation approach required for scaling large datasets and training Deep Learning models.